# Static

## Methods, Attributes, FastJson

Yegor Bugayenko

Lecture #2 out of 8
80 minutes

The slidedeck was presented by the author in this YouTube Video.

All visual and text materials presented in this slidedeck are either originally made by the author or taken from public Internet sources, such as web sites. Copyright belongs to their respected authors.

- Pre-Test
- Methods
- Attributes
- FastJson as Example

Chapter #1:

# Pre-Test

# How Many Static “Entities” Do You See Here?

```
class FigureUtils {
  static final float PI =
  3.1415926;
  static final FigureUtils INSTANCE
  ;
  static {
    INSTANCE = new FigureUtils();
  }
  private FigureUtils() { /* empty
  */ }
  static float perimeter(Circle c)
  {
    return 2 * c.radius * PI;
  }
}
```

How many?

- three
- four
- five
- six
- maybe seven

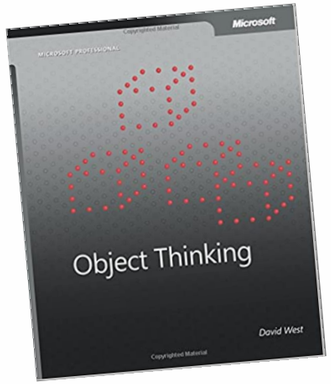
# Chapter #2:
# Methods

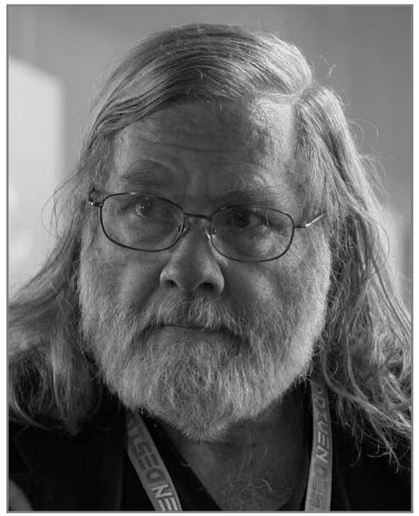
# What Static Methods Are For?

```
1 class Circle {
2   public float radius;
3 }
4
5 class GeometryUtils {
6   static float calcArea(Circle c) {
7     return c.radius * c.radius *
    3.14;
8   }
9 }
```

```
1 class Circle {
2   public float radius;
3   float area() {
4     return radius * radius * 3.14;
5   }
6 }
```

Most notable Java examples [5]: FileUtils, IOUtils, and StringUtils from Apache Commons; Files from JDK 7; Iterators from Google Guava.



"Traditional computer thinking treats data as passive and helpless, manipulated and moved by active procedures. Procedures tend to be egocentric and think that all data, its definitions and its values, are properly determined by the procedure currently using that data. The result is data that is inconsistent in definition, inconsistent in value, inaccessible because some other procedure has possession, or needlessly duplicated."

— David West. *Object Thinking*. Pearson Education, 2004. DOI: 10.5555/984130

# What's Wrong with "Utils"?

1) They are unbreakable dependencies

2) They are eager, not lazy

3) They are not cohesive

# 1) Tight Coupling

```
1 void paintIt(Circle c) {
2   float s = GeometryUtils.calcArea(
    c);
3   float p = s * 5.55;
4   // paint it using the "p"
5 }
```

```
1 void paintIt(Circle c) {
2   float s = c.area();
3   float p = s * 5.55;
4   // paint it using the "p"
5 }
```

Which snippet is easier to test? Try to write a test for the first one, expecting `s` to be equal to `42.0` [2].

# 2) Imperative, Not Declarative

```
1 void paintIt(Circle c) {
2   float s = GeometryUtils.calcArea(
    c);
3   if (t) { return; }
4   float p = s * 5.55;
5   // paint it using the "p"
6 }
```

```
1 void paintIt(Circle c) {
2   float s = new AreaOf(c);
3   if (t) { return; }
4   float p = s * 5.55;
5   // paint it using the "p"
6 }
```

Which snippet is more eager to calculate the area of the circle? Which one does it when it's really necessary? [1]

# 3) Low Cohesion

```
class GeometryUtils {
  static float calcArea(Circle c);
  static float calcPerimeter(Circle
   c);
  static float calcSinus(Angle a);
  static float calcCosinus(float s)
  ;
  // and many more...
}
```

```
class Circle {
  float area();
  float perimeter();
}
class Angle {
  float sinus();
}
class Float {
  float cosinus();
}
```

Which class looks more cohesive to you, the utility class `GeometryUtils` or the `Circle`?

# Chapter #3:
# Attributes

# Public Literals

```
class Constants {
  public static float PI =
  3.1415926;
  public static String UTF_8 = "utf
  -8";
  public static String LOCALE = "fr
  ";
  // and many more
}

println("S'il vous plaît",
  Constants.LOCALE);
printf("It is %see speech!",
  Constants.LOCALE);
```

```
class Print { }
class TextInFrench { }

new Print(
  new TextInFrench(
    "S'il vous plaît"
  )
)
```

We must solve the problem of functionality duplication, not just data duplication [3].

# Singletons

```
1 class Canvas {
2   public static Canvas INSTANCE =
3     new Canvas();
4   private Canvas() {}
5   public void addCircle(Circle c);
6 }
7
8 Canvas.INSTANCE.addCircle(c1);
9 Canvas.INSTANCE.addCircle(c2);
```

```
1 c = new Canvas();
2 c.addCircle(c1);
3 c.addCircle(c2);
```

Forget about singletons; never use them. Turn them into dependencies and pass them from object to object through the operator `new` [4].

Chapter #4:

# FastJson as Example

[ JSONPath ]

# JSONPath

```java
public int size(Object rootObject) {
    if (rootObject == null) {
        return -1;
    }

    init();

    Object currentObject = rootObject;
    for (int i = 0; i < segments.length; ++i) {
        currentObject = segments[i].eval(this, rootObject, currentObject);
    }

    return evalSize(currentObject);
}
```

```java
public static int size(Object rootObject, String path) {
    JSONPath jsonpath = compile(path);
    Object result = jsonpath.eval(rootObject);
    return jsonpath.evalSize(result);
}
```

Some mind-blowing statistics:

- 3 constructors
- 1 interface implemented
- 59 methods at `JSONPath`
- 4,363 lines of code
- 34 inner static classes
- 74 times `static` keyword

https://github.com/alibaba/fastjson/blob/master/src/main/java/com/alibaba/fastjson/JSONPath.java

[ JSONPath ]

# References


[1] Yegor Bugayenko. *Composable Decorators vs. Imperative Utility Methods*. https://www.yegor256.com/150226.html. [Online; accessed 08-07-2024]. Feb. 2015.

[2] Yegor Bugayenko. *OOP Alternative to Utility Classes*. https://www.yegor256.com/140505.html. [Online; accessed 08-07-2024]. May 2014.

[3] Yegor Bugayenko. *Public Static Literals ... Are Not a Solution for Data Duplication*. https://www.yegor256.com/150706.html. [Online; accessed 08-07-2024]. July 2015.

[4] Yegor Bugayenko. *Singletons Must Die*. https://www.yegor256.com/160627.html. [Online; accessed 08-07-2024]. June 2016.

[5] Yegor Bugayenko. *Utility Classes Have Nothing to Do With Functional Programming*. https://www.yegor256.com/150220.html. [Online; accessed 08-07-2024]. Feb. 2015.

[6] David West. *Object Thinking*. Pearson Education, 2004. DOI: 10.5555/984130.